efrei
INGÉNIEURS - NUMÉRIQUE
PARIS PANTHÉON-ASSAS UNIVERSITÉ

LA GRANDE ÉCOLE D'INGÉNIEURS GÉNÉRALISTES
ET D'EXPERTS DU NUMÉRIQUE
DE L'UNIVERSITÉ PANTHÉON-ASSAS

Programmation Web

Javascript

Mohamed HAMIDI
Maha Naceur
2025 - 2026

Plan

Javascript : les bases

Les structures de contrôle

Les fonctions et Les tableaux

Les évènements

Javascript : un peu plus loin

Fonctions anonymes & fonction fléchées

Les objets itérables

Les objets en Javascript

Le format JSON

Plan

Javascript : les bases

Les structures de contrôle

Les fonctions et Les tableaux

Les évènements

Javascript : un peu plus loin

Fonctions anonymes & fonction fléchées

Les objets itérables

Les objets en Javascript

Le format JSON

Javascript : qu’est ce que c’est ?

Abrégé en «JS», c’est un langage de script léger, principalement connu comme le langage de scripts des pages Web.

C’est un langage dynamique;

Le Code est principalement interprété par le navigateur client (interprétation dépendante du type et de la version du navigateur).

Langage événementiel (association d’actions aux événements déclenchés par l’utilisateur (passage de souris, clic, saisie clavier, etc…).

Utilisé aussi dans de nombreux environnements extérieurs aux navigateurs web tels que Node.js, Apache CouchDB voire Adobe Acrobat.

Javascript

Avantages :

Supporté (par défaut) par les principaux navigateurs, ne nécessite pas de plug-in particulier.

Accès aux objets contenus dans un document HTML (+ possibilité de les manipuler relativement facilement).

Possibilité de mettre en place des animations

Inconvénients :

Les utilisateurs peuvent empêcher l'exécution de code JavaScript, souvent en raison des erreurs générées par les scripts, ouencore en raison de la nature de l'interaction (apparition de nouvelles fenêtres, ...).

Lenteur d'exécution des scripts, ainsi que pour les scripts complexes, un certain délai avant le démarrage.

Exécution de Javascript

Il n'y a pas de **"main method"**

- Le script est exécuté de haut en bas

Il n'y a pas de **compilation** par le développeur

- JavaScript est compilé et exécuté à la volée par le navigateur

- **C'est le seul langage disponible côté navigateur**
- Mais est aussi disponible côté serveur via NodeJS

Où se trouve le Javascript ?

1- Directement dans la balise ouvrante d'un élément HTML ;

Exemple1Javascript.html

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>TP 1</title>
    </head>
    <body>
        <h1> Premier exemple Javascript </h1>
        <button onclick = "alert('Bonjour SRT3 ! ')"> Dites Bonjour </button>

    </body>
</html>
```

2- Dans un élément script, au sein d'une page HTML ;

```html
<html>
    <head>
        <meta charset="utf-8">
        <title>TP 1</title>
        <link rel="stylesheet" href="monstyle.css">
        <script>
            document.addEventListener('DOMContentLoaded', function(){

            let bonjour = document.getElementById('b1');
            bonjour.addEventListener('click', alerte);
            function alerte(){
                alert('Bonjour SRT3 !');
            }
        });
        </script>
    </head>
    <body>
        <h1> Premier exemple Javascript </h1>
        <button id = 'b1'> Dites Bonjour </button>

    </body>
</html>
```

3- Dans un fichier séparé contenant exclusivement du JavaScript et portant l'extension .js.

Exemple3Javascript.html | ExempleJs.js

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8">
        <title>TP 1</title>
        <link rel="stylesheet" href="monstyle.css">
        <script src='ExempleJs.js' async ></script>

    </head>
    <body>
        <h1> Premier exemple Javascript </h1>
        <button id = 'b1'> Dites Bonjour </button>

    </body>
</html>
```

Exemple3Javascript.html | ExempleJs.js

```javascript
let bonjour = document.getElementById('b1');

bonjour.addEventListener('click', alerte);

function alerte(){
    alert('Bonjour SRT3 !');
}
```

Javascript : la syntaxe

1. JavaScript est sensible à la casse.
2. En JavaScript, les instructions sont séparées par des points virgules.
3. S'il y'a plusieurs déclarations sur une seule ligne, alors elles doivent être séparées par un point virgule.
4. Les commentaires :

```
// un commentaire sur une ligne

/* un commentaire plus
   long sur plusieurs lignes
 */

/* Par contre on ne peut pas /* imbriquer des commentaires */ SyntaxError */
```

Javascript : Déclaration et initialisation de variables

Pour déclarer une variable en JavaScript, deux possibilités :

- Le mot clef « var » : déclarer une variable et éventuellement initialiser sa valeur
- Le mot clef « let » : déclarer une variable dont la portée est celle du bloc courant, éventuellement en initialisant sa valeur.

suivi du nom qu'on souhaite donner à notre variable.

On peut également définir des constante en utilisant le mot clef « const »

```
var mois = 4; //mois stocke un entier
var jour = 'Novembre'; //jour stocke une Chaine de caractère
var bool = true; //bool stocke un booléen
const PI = 3.14; //constante
```

Rqs :

1. Une variable déclarée grâce à l'instruction var ou let sans valeur initiale définie vaudra undefined
2. Les noms des variables sont appelés **identifiants**.
3. Un identifiant JavaScript doit commencer par une lettre, un tiret bas (_) ou un symbole dollar ($).

let vs var

```js
let x = 1;

if (x === 1) {
  let x = 2;

  console.log(x);
  // expected output: 2
}

console.log(x);
// expected output: 1

```

```
> 2
> 1
```

```js
var x = 1;

if (x === 1) {
  var x = 2;

  console.log(x);
  // expected output: 2
}

console.log(x);
// expected output: 2

```

```
> 2
> 2
```

Opérateurs, concaténation et affichage

1. **Les opérateurs : +, -, /, *, %, == (valeurs), === (valeurs et types), !=(valeur), !==(valeurs ou types), <, >, etc**

```
var x = 1;
var y = 3;
var z = x+y;//Addition
var t = x-z; //Soustraction
x += 1; //incrémentation de 1
```

2. **Concaténation**

```
var mois = 4;
var jour = 'Novembre';
var DateTP1 = mois + jour + '2020';
```

3. **Affichage**

```
var x = 1;
var y = 3;
alert('x =' + x + '\n y = ' + y);
alert(`x = ${x}
    y =  ${y}
    Leur somme vaut ${x+y}`);
```

Structures de contrôle et boucles

4. Structures conditionnelles simples

```
if(x>1){
    alert('La valeur de x est supérieure à 1');
}else if(x == 1){
    alert('La valeur de x est 1');
}else{
    alert('La valeur de x est inférieure à 1');
}
```

```
var x = 1;

if(x>1){
    alert('La valeur de x est supérieure à 1');
}else{
    alert('La valeur de x est inférieure à 1');
}
```

5. Structure conditionnelle multiple

```
switch(x){
    case 0 : alert('La valeur de x est supérieure à 1');
    break;
    case 1 : alert('La valeur de x est 1');
    break;
    case 2 : alert('La valeur de x est inférieure à 1');
    break;
    default : alert("Je ne sais pas !");

}
```

6. Les boucles

```
var x = 0;
while(x < 10){
    x++;
}

do{
    x++;
}while(x<10);

for(let i = 0; i<10; i++){
    x++;
}
```

Opérateurs === et !==

=== **et** **!==** sont les véritables opérateurs de comparaison,
toujours les utiliser!

```
'' === '0'// false
'' === 0  // false
0 === '0'  // false
NaN === NaN  // still weirdly false
[''] === ''  // false
false === undefined  // false
false === null  // false
null === undefined  // false
```

```
0===‘0’// false    0==‘0’ true
```

Tableaux

Les tableaux sont les objets pour définir des listes

```
// Creates an empty list
const list = [];
const groceries = ['milk', 'cocoa puffs'];
groceries[1] = 'kix';

// For each loop
for (let item of groceries) {
  console.log(item);
}
```

Les fonctions

On peut créer une fonction grâce à « function »

```
function aleatoire(){
    return Math.random()*100;
}

function Multiplication(nombre1, nombre2){
    return nombre1 + '*' + nombre2 + '=' + (nombre1 * nombre2);
}

document.getElementById('p1').innerHTML = aleatoire();
document.getElementById('p2').innerHTML = Multiplication(5,10);
```

Une fonction peut être anonymes (ne pas avoir de nom correspondant).

```
var carré = function (n){
    return n * n;
}

var x = carré(4);
```

Rq : La déclaration d'une fonction peut être faite après l'appel

Les événements

- JavaScript = langage réactif
- L'interaction avec l'utilisateur est gérée via des événements
- Événement = action ou occurrence qui se produit dans le système et dont le système vous informe afin d'y répondre,

Les événements sont généralement des noms d'attributs HTML, le contenu de ces attributs sera du JavaScript.

De manière générale, JavaScript permet d'associer une instruction JavaScript à chaque événement : soit l'appel à une fonction de base JavaScript, soit l'appel à l'une de nos propres procédures.

Les événements détectables

onChange : la valeur d 'un champ de formulaire à été modifiée par l'utilisateur

onClick : un clic souris est déclenché sur un objet

onLoad :la page est chargée par le navigateur

onMouseover : la souris est déplacée sur un objet

onSelect : un champ de formulaire est sélectionné (par clic souris ou tabulation)

onSubmit : un formulaire est soumis

onUnload : l'utilisateur quitte la page

Les fonctions associées à une action de l'utilisateur

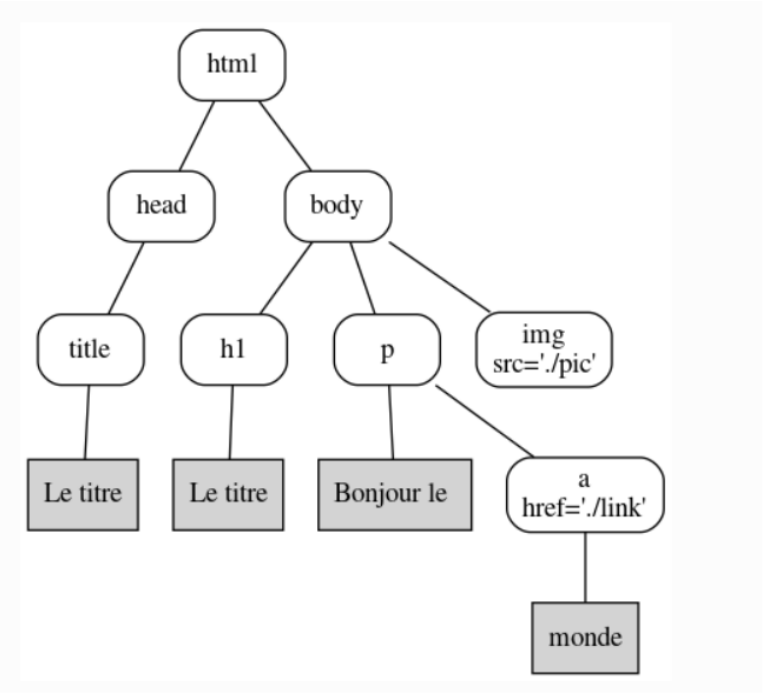
Quelques exemples :

- **alert** : ouvre une fenêtre avec un message donné,
- modifier **window.location** pour aller vers une autre page,
- **document.write**: écrit un texte donné,
- **setTimeout** et **setInterval** permettent respectivement de différer et de répéter une instruction ;
- **focus** : permet d'activer un élément de la page (comme la case à remplir obligatoirement dans un formulaire), **blur** de le désactiver.

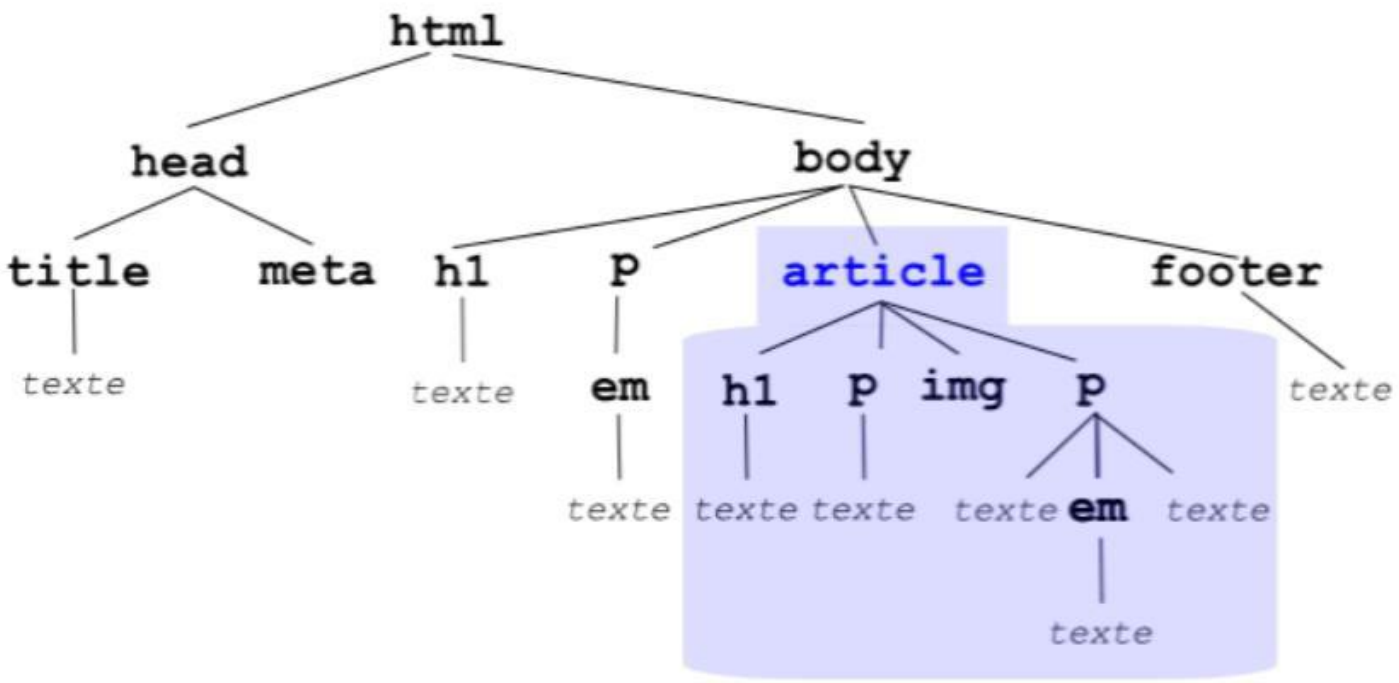
```
// déclenchement d'une alerte immédiate (alert)
<button onclick="alert('Coucou !')">cliquez ici !<button>

// déclenchement d'une alerte dans une demi-seconde (setTimeout)
<button onclick="setTimeout('alert(\'Surprise !\')',500)">
cliquez ici !<button>

// répétition toutes les 5 secondes (setInterval)
var clock_id = setInterval('alert(\'hi hi !\');',5000);
...
clearInterval(clock_id); // fin de la répétition
```

Document Object Model

- ✓ **DOM** : représentation structurée du document sous forme d'un arbre.
- ✓ DOM : définit la façon dont la structure peut être manipulée.
- ✓ JavaScript interagit avec la page via la structure DOM accessible via une variable appelée « document »
- ✓ DOM : accéder à un ou plusieurs éléments
- ✓ Pour accéder à un élément par son « id », utiliser « GetElementById »
- ✓ Si un élément HTML dispose d'un « id », une variable est créée pour lui.



DOM vu comme un arbre



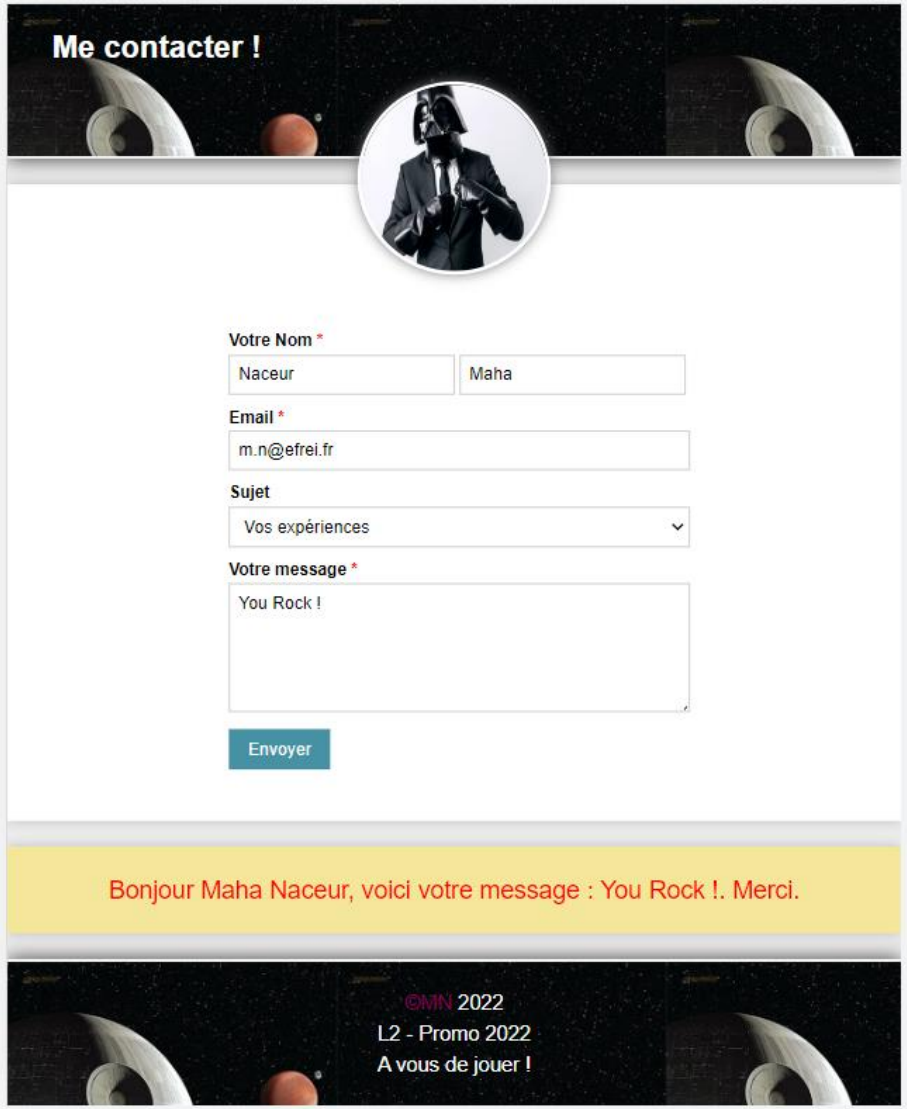
- Nœud article: sous arbre de la racine
- Nœuds du sous arbre : descendants du nœud article
- Nœud article : père des nœuds h1, p, img et p
- h1, p, img et p : fils du nœud article
- Un nœud texte ne contient qu'une chaîne de caractères. Il peut ne pas avoir d'enfants et est toujours une feuille de l'arbre.

DOM : accès à un élément

- Par son attribut **id**
- Par son attribut **class**
- Par sa balise
- Par un sélecteur **CSS**
- Par son attribut **name**

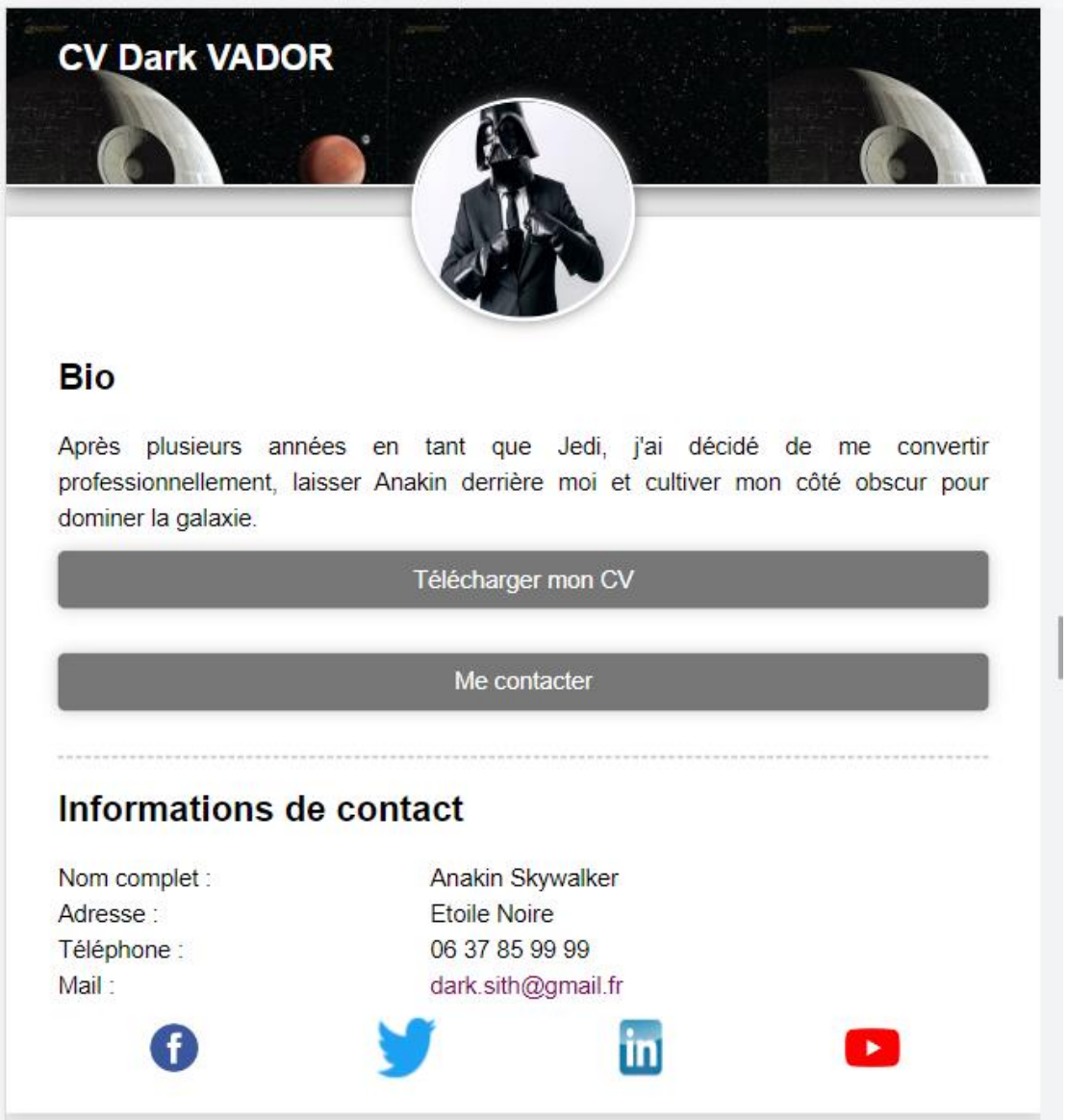
Fonction	Description
getElementById(id)	Renvoie un élément par son id
getElementsByTagName(balise)	Renvoie la liste des éléments du document dont la balise est « balise »
createTextNode(texte)	Crée une balise texte
createElement(balise)	Crée une balise HTML
getElementsByClassName(className)	Renvoie la liste des éléments par nom de classe

Livecoding : Un message à Dark Vador



Codepen(sans CSS) : ici

Exemple complet : CV de Dark Vador



Code complet sur Moodle

Plan

Javascript : les bases

Les structures de contrôle

Les fonctions et Les tableaux

Les évènements détectables

Javascript : un peu plus loin

Gestion de l'asynchrone

Fonctions anonymes & fonction fléchées

Les objets itérables

Les objets en Javascript

Le format JSON

Synchrone Vs Asynchrone

Synchrone

En informatique, on dit que deux opérations sont **synchrones** lorsque la seconde attend que la première ait fini son travail pour démarrer (**dépendance**).

Asynchrone

Deux opérations sont qualifiées d'**asynchrones** en informatique lorsqu'elles sont **indépendantes** c'est à dire lorsque la deuxième opération n'a pas besoin d'attendre que la première se termine pour démarrer.

L’asynchrone en Js

- En JS, chaque ligne de code est exécutée de façon **synchrone**, mais il est possible de demander à exécuter du code de manière asynchrone càd les actions peuvent être exécutées indépendamment du flux d'exécution principal
- En JS, les opérations **asynchrones** sont placées dans des files d’attentes qui vont s’exécuter après que le fil d’exécution principal ou la tâche principale (le « main thread » en anglais) ait terminé ses opérations => Elles ne bloquent donc pas l’exécution du reste du code JavaScript
- Exemple de fonctions asynchrones : setTimeout, setInterval, setImmediate

Exemple : setTimeout

- c’est la fonction la plus répandue lorsque l'on veut exécuter du code asynchrone sans bloquer le fil d'exécution en cours.
- Cette fonction prend 2 paramètres :
  - La fonction à exécuter de manière asynchrone (qui sera donc ajoutée à la file d'attente de l'event loop) ;
  - Le délai, en millisecondes, avant d'exécuter cette fonction.

```
setTimeout(function() {
    console.log("I'm here!")
}, 5000);

console.log("Where are you?");
//le texte  Where are you?  s'affichera avant  I'm here! ,
//qui ne sera affiché qu'au bout de 5 secondes.
```

Exemple : addEventListener()

Un gestionnaire d'événement est une forme de programmation asynchrone
Un gestionnaire d'événements est appelé au moment où l'événement se produit

La méthode addEventListener() de EventTarget attache une fonction à appeler chaque fois que l'évènement spécifié est envoyé à la cible. Les cibles sont :

- Un objet de la classe Element
- L'interface Document
- Un objet window
- Tout objet prenant en charges les événements

La méthode agit en ajoutant une fonction ou un objet qui implémente l'objet EventListener qui gère un événement diffusé par un objet EventTarget.
Elle l'ajoute à la liste des gestionnaires d'événement pour le type d'événement spécifié sur la cible à partir de laquelle il est appelé

Exemple (codepen)

Les Callbacks

Les fonctions peuvent être passées en paramètres d'autres fonctions

```
const groceries = ['milk', 'cocoa puffs'];

// For each loop
for (let item of groceries)  console.log(item);

// Callback style  groceries.forEach(function(item)
                                    {
  console.log(item);
});

// Nerdy callback style (use that!) groceries.forEach(item
=> {
  console.log(item);
});
```

L’API Fetch

L’API Fetch comporte une seule fonction, concise et facile à utiliser

```
fetch('images.txt');
```

- Fetch prend en paramètre l’URL de la ressource que vous voulez consulter
- Elle retourne une "Promise"

Les « Promise »

```
const promise = fetch('images.txt');
promise.then(onSuccess, onFail);
```

Une promesse peut être dans un des états suivants

- **pending**: état initial, promesse non-exécutée
- **fulfilled**: la promesse a été exécutée correctement
- **rejected**: la promesse a rencontrée une erreur

On attache un gestionnaire à une promesse via `.then()`

```
fetch('images.txt').then(response =>
console.log(response.status));
```

Les évènements dans Javascript

Afin de réagir lors d'un clic sur un élément, il faut *écouter* cet événement. Pour cela, nous avons à notre disposition la fonction **addEventListener**(). Cette fonction nous permet d'écouter tous types d'événements (pas que le clic)

→ Réagir à un événement, c'est faire une action lorsque celui-ci se déclenche. Écouter, c'est vouloir être averti quand l'événement se déclenche.

```
addEventListener(<event>, <callback>)
```

Exemples des évènements

Evènement	Se produit	S'applique à
onload	lors du chargement.	Objet javascript : image, window, frame img, body, frameset, frame
onmouseover	quand le curseur passe au dessus de l'objet	Objet javascript : link, area Balises HTML : a, area
onchange	quand on change le contenu.	Objet javascript : form Balises HTML : input, textarea,select
onclick	quand on clique.	Objet javascript : link,document, form Balises HTML : a, body,area
onselect	quand on sélectionne.	Objet javascript : form Balises HTML : input, textarea
ondragdrop	quand on déplace un objet dans une fenêtre.	Objet javascript : window, frame Balises HTML : body, frame, frameset
onerror	lorsqu'il se produit une erreur de script.	Objet javascript : image, window, frame Balises HTML : img, body, frame, frameset
onunload	quand on ferme la fenêtre.	Objet javascript : window, frame Balises HTML : body, frameset, frame
onsubmit	quand on envoie un formulaire.	Objet javascript : form Balises HTML : form
onmousemove	quand le curseur bouge.	Objet javascript : link, document, form Balises HTML : a, body, form
onreset	quand on réinitialise.	Objet javascript : form Balises HTML : form

Exemple d’évènement

```
1 const elt = document.getElementById('mon-lien');    // On récupère l'élément sur lequel on veut
  détecter le clic
2 elt.addEventListener('click', function() {          // On écoute l'événement click
3     elt.innerHTML = "C'est cliqué !";               // On change le contenu de notre élément pour
  afficher "C'est cliqué !"
4 });
```

preventDefault()

```
1 const elt = document.getElementById('mon-lien');    // On récupère l'élément sur lequel on veut
  détecter le clic
2 elt.addEventListener('click', function(event) {     // On écoute l'événement click, notre callback
  prend un paramètre que nous avons appelé event ici
3     event.preventDefault();                         // On utilise la fonction preventDefault de notre
  objet event pour empêcher le comportement par défaut de cet élément lors du clic de la souris
4 });
```

stopPropagation()

Récupérer des données utilisateur avec les évènements

Connaître la position de la souris → Récupérer les données des évènements

Détecter le mouvement de la souris: Exemple

```
1 elt.addEventListener('mousemove', function(event) {
2     const x = event.offsetX; // Coordonnée X de la souris dans l'élément
3     const y = event.offsetY; // Coordonnée Y de la souris dans l'élément
4 });
```

Pour plus d'information sur l'évènement MouseMove

Plus de connaissances sur l' Interface Event

Lire le contenu d'un champ de texte

```html
<label>Choose an ice cream flavor:
    <select id="ice-cream" name="ice-cream">
        <option value="">Select One …</option>
        <option value="chocolate">Chocolate</option>
        <option value="strawberry">Strawberry</option>
        <option value="vanilla">Vanilla</option>
    </select>
</label>
```

Icecream.html

Icecream.js

```javascript
document.addEventListener('DOMContentLoaded',function() {
    document.querySelector('select[name="ice-cream"]').onchange=changeEventHandler;
},false);

function changeEventHandler(event) {
    // You can use “this” to refer to the selected element.
    if(!event.target.value) alert('Please Select One');
    else alert('You like ' + event.target.value + ' ice cream.');
}
```

Récupérer n'importe quelle balise

- Nous avons vu spécifiquement comment récupérer les champs d'un formulaire
  - Il y a une méthode plus générique pour accéder aux éléments du DOM
- Si on met un attribut id à une balise, on peut la récupérer avec **document.getElementById**:

```
<input id="my_input" type="text" name="login">
<script>
    let my_input = document.getElementById('my_input')
    console.log(my_input.value)
</script>
```

- On peut effectuer une opération similaire avec **document.querySelector()**, qui permet de chercher une balise spécifique avec toutes les possibilités qu'offrent les sélecteurs CSS:

```
let my_input = document.querySelector('#my_input’)
```

  - Notez qu'on a rajouté un # avant my_input dans le dernier exemple

Modifier un texte

- Pour modifier le contenu textuel d'une balise comme **p** ou **span**, on peut utiliser la propriété textContent:

```
<p id="paragraphe"></p>
<script>
    var p = document.querySelector('#paragraphe')
    p.textContent = 'Du texte ajouté dynamiquement'
</script>
```

- **Une autre propriété innerHTML existe, mais évitez tant que possible de l'utiliser, car elle est très dangereuse en termes de sécurité !**

Les fonctions anonymes et les fonctions fléchées

- Une expression de fonction fléchée (arrow function en anglais) permet d'avoir une syntaxe plus courte que les expressions de fonction
- Il est possible de définir une fonction directement à l'intérieur d'une autre fonction.

```
const message = function() {
    console.log("This message is shown after 3 seconds");
}

setTimeout(message, 3000);
```

Fonction anonyme

```
setTimeout(function() {
    console.log("This message is shown after 3 seconds");
}, 3000);
```

```
setTimeout(() => {
    console.log("This message is shown after 3 seconds");
}, 3000);
```

Fonction fléchée

Les fonctions de rappel : callbacks

Une fonction "callback" est une fonction passée en argument d'une fonction asynchrone et qui sera appelée dès la fin de l'exécution de celle ci.

```
const message = function(){
    console.log("This message is shown after 3 seconds");
}
setTimeout(message, 3000);
```

Quand on passe une fonction en paramètre, on donne juste son nom, sans les parenthèses (la présence de parenthèses provoque un appel de fonction)

Cela peut se transformer en cauchemar (callback hell) si vous avez plusieurs callbacks où chacun nécessite le résultat de l'opération précédente.

Exemple : callback

JS

```javascript
// function
function greet(name, callback) {
    console.log('Hi' + ' ' + name);
    callback();
}

// callback function
function callMe() {
    console.log('I am callback function');
}

// passing function as an argument
greet('Efrei', callMe);
```

Codepen ici

Console

```
"Hi Efrei"
"I am callback function"
```

Exemple :

JS

```js
let utilisateur = {
    prenom: 'Maha',
    nom: 'Naceur',
    age: 37,

    //Méthode itérateur avec Symbol.iterateur comme clef
    [Symbol.iterator](){
        //Renvoie un tableau contenant les valeurs des propriétés de l'objet
        let tableau = Object.values(this);
        let prop = 0;

        return{
            next(){
                if(prop < tableau.length){
                    return{
                        value: tableau[prop++],
                        done: false
                    };
                }else{
                    return{
                        value: undefined,
                        done: true
                    };
                }
            }
        };
    }
};

for (let p of utilisateur) {
    alert(p);
}
```

Codepen ici

Les objets Javascript

- Javascript utilise le paradigme « objets »
- Un objet Javascript est une entité à part entière qui possède des propriétés et un type
- L'environnement JS fournis des objets (prédéfinis) natifs (Date, String, Math, etc…), mais possibilité de construire de nouveaux objets
- Un objet est défini grâce à 3 éléments :
  - **son constructeur,**
  - **ses propriétés**
  - **ses méthodes** (fonctions spécifiques à l'objet)

Création de l'objet en appelant le constructeur sans lui donner de paramètres

Création de l'objet en appelant le constructeur avec le Timestamp en paramètre

```
1  var aujourdhui = new Date();
2      document.write("Date d'aujourdhui : " + aujourdhui);
3      document.write ("<br>"); //saut de ligne
4      var autredate =new Date(24*60*60*1000);
5      document.write("Autre date : " + autredate);
6      var madatenaissance = new Date(2022,05,13) ;
7      document.write ("<br>"); //saut de ligne
8      document.write ("Ma date de naissance : " + madatenaissance);
9      document.write("<br>" + "timestamp de l'instant présent :" + aujourdhui.getTime());
10     document.write("<br>" + "timestamp de ma date de naissance :" + madatenaissance.getTime());
```

Création de l'objet en appelant le constructeur avec les valeurs d'une date dans l'ordre

Méthode associée aux objets de type Date

Les Objets

Collection de paires clé - valeur, peut être utilisé comme une hashmap

```
const prices = {};  const
scores =       {
  peach: 100,
  mario: 88,
  luigi: 91
};
```

```
console.log(scores['peach']);                // 100
console.log(scores.peach);  scores.peach// 100
=              20;
console.log(scores.peach);              // 20
```

Ajout d'une fonction sur un objet

On peut ajouter une fonction dans un objet, `this` désigne l'objet receveur

```
const scores = { peach: 100,
  mario: 88,
  luigi: 91,
  total: function() {
    return this.peach + this.mario + this.luigi;
  }
};
```

Classes ES6

```
class Person {  constructor(name, age)       {
    this.name = name; this.age = age;
  }

  displayAge() {  console.log(this.age);
  }
  displayName() {  console.log(this.Name);
  }
  displayAgeAndName() { this.displayAge();
    this.displayName();
  }
}

const p = new Person("Joe",1);
```

- Le constructeur est optionnel
- Tous les attributs et les méthodes sont publics
- Tous les appels de méthodes et les accès d'attributs se font via `this`

L'objet itérable : Array

```
let etudiants = ['Maha', 'Nathalie', 'Florian'];
console.log(etudiants.length);
//3
```

- Objet global utilisé pour créer des tableaux
- Les objets Array possèdent des méthodes permettant de parcourir et de modifier leurs contenus

```
let first = etudiants[0];
//Maha
let second = etudiants[2];
//Florian
```

- L'accès à un élément du tableau se fait via son index

```
etudiants.forEach(function(item, index, array) {
    console.log(item, index);
  });
// Maha 0
// Nathalie 1
// Florian 2
```

- Boucler sur le contenu du tableau

```
let nouveau = etudiants.push('Zouhour');
let last = etudiants.pop();
let first = etudiants.shift();
```

- Ajouter Fin supprimer Début/Fin un élément

Créons nos propres objets

Définition d'un nouvel objet

```
let maClasse = {
    promo: 2024,
    annee: "2021-2022",
    intitule : "L2/L2-R/L2-INT/L2-Bdx"
  };
```

Valeur attribuée à cette propriété

Propriété de l'objet

Autres syntaxes

Ou en utilisant le constructeur

mot-clé this pour affecter des valeurs aux propriétés d'un objet en fonction des valeurs passées en arguments de la fonction.

```
function Classe(promo, annee, intitule) {
  this.promo = promo;
  this.annee = annee;
  this.intitule = intitule;
}
let maClasse = new Classe(2024, "2021-2022", "L2/L2-R/L2-INT/L2-Bdx");
```

Créons nos propres objets

Ou en utilisant la méthode [Object.create]

Propriétés avec des valeurs initiales

Méthode permettant d'afficher l'année ans la console

Définition de deux objets de type Classe

```
let Classe ={
    promo : 2024,
    annee : "2021-2022",
    intitule : "L2/L2-R/L2-INT/L2-Bdx",
    afficherIntitule : function(){
        console.log(this.annee);
    }

}

let groupeA = Object.create(Classe);
let groupeBdx = Object.create(Classe);
groupeA.intitule = "L2";
groupeBdx.intitule = "LE-Bdx";
groupeA.afficherIntitule();
groupeBdx.afficherIntitule();
```

Attribution d'une nouvelle valeur à la propriété « intitule » propre à chacun des objets créés

Affichage de l'intitule de chaque objet en utilisant la méthode afficherIntitule()

Créons nos propres objets

Pour créer un objet, on commence toujours par la définition et l'initialisation d'une variable => Vous pouvez tester en ouvrant la console Javascript de votre navigateur et tapez l'identifiant de l'objet que vous créez.

Object-Js.js

```
var Classe = {}
```

Object-Js.html

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4       <meta charset="utf-8">
5       <script type="text/javascript" src="Objects-Js.js"></script>
6   </head>
7   <body>
8
9   </body>
10  </html>
```

```
> Classe
<· ▼{} i
    ▼[[Prototype]]: Object
      ▶constructor: ƒ Object()
      ▶hasOwnProperty: ƒ hasOwnProperty()
      ▶isPrototypeOf: ƒ isPrototypeOf()
      ▶propertyIsEnumerable: ƒ propertyIsEnumerable()
      ▶toLocaleString: ƒ toLocaleString()
      ▶toString: ƒ toString()
      ▶valueOf: ƒ valueOf()
      ▶__defineGetter__: ƒ __defineGetter__()
      ▶__defineSetter__: ƒ __defineSetter__()
      ▶__lookupGetter__: ƒ __lookupGetter__()
      ▶__lookupSetter__: ƒ __lookupSetter__()
       __proto__: (...)
      ▶get __proto__: ƒ __proto__()
      ▶set __proto__: ƒ __proto__()
```

Créons nos propres objets

```
var etudiant = {
    nom: ['Maha', 'Naceur'],
    age: 37,
    sexe: 'féminin',
    interets: ['musique', 'films'],
    bio: function() {
      alert(this.nom[0] + ' ' + this.nom[1] + ' a ' + this.age + ' ans. aime ' + this.interets[0] + ' et ' + this.interets[1] + '.');
    },
    salutation: function() {
      alert('Bonjour ! Je suis ' + this.nom[0] + '.');
    }
  };
```

```
> etudiant.nom
<· ▶ (2) ['Maha', 'Naceur']
> etudiant
<· ▼ {nom: Array(2), age: 37, sexe: 'féminin', interets: Array(2), bio: ƒ, …}
      age: 37
    ▶ bio: ƒ ()
    ▶ interets: (2) ['musique', 'films']
    ▶ nom: (2) ['Maha', 'Naceur']
    ▶ salutation: ƒ ()
      sexe: "féminin"
    ▶ [[Prototype]]: Object
```

Ajouter/supprimer des propriétés

AJouter une propriété pour toutes les instances d'un type d'objet déjà créé

```
Classe.prototype.nombre = 0;
groupeA.nombre = 36;
groupeBdx.nombre = 34;
```

Prototype permet de définir une propriété qui sera partagée par tous les objets d'un même type plutôt qu'elle ne soit définie que pour un seul objet

Supprimer une propriété

```
delete Classe.nombre
```

Getteurs/Setteurs

- Un accesseur (*getter*) est une méthode qui permet de récupérer la valeur d'une propriété donnée.
- Un mutateur (*setter*) est une méthode qui permet de définir la valeur d'une propriété donnée.

- Les accesseurs et mutateurs peuvent être :
  - définis avec les initialiseurs d'objet, ou
  - ajoutés après la construction en utilisant une méthode pour ajouter un accesseur ou un mutateur.

```javascript
let Classe = {
    promo : 2024,
    annee : "2021-2022",
    intitule : "L2/L2-R/L2-INT/L2-Bdx",
    nombre : 0,
    afficherIntitule : function(){
        console.log(this.annee);
    },
    get nombreEtduiants() {
        return this.nombre;
      },
    set majNombreEtudiant(x) {
        this.nombre = this.nombre + x;
      }
  };

  console.log(Classe.nombre); // 0
  console.log(Classe.nombreEtduiants); // renvoie 0 également
  o.majNombreEtudiant = 3;
  console.log(Classe.nombre); // 50
```

A voir aussi:
Object.defineProperties() (ou Object.defineProperty()

Toujours des objets !!!

À chaque fois que vous avez travaillé avec un exemple qui utilise une API ou un objet JavaScript natif, nous avons utilisé des objets

Exemples :

- maChaine.split(‘,’); : instance de type String
- document.createElement(‘div’) : instance de la classe Document

Manipulation des données JSON

- JSON : JavaScript Object Notation
- Format standard utilisé pour représenter des données structurées de façon semblable aux objets JavaScript
- Popularisé par Douglas Crockford
- Peut être utilisé dans de nombreux autres langages de programmation

- JSON : chaîne de caractères qui est converti en un objet Javascript natif (parsage)
- JS fournit un objet global JSON disposant des méthodes pour assurer la conversion

- Un objet JSON peut être stocké dans un fichier avec l'extension .json et le MIME Type application/json
- Permet de hiérarchiser les données

Exemple de fichier JSON

```json
{
    "promo": "2021-2022",
    "annee": 2024,
    "nombre": 30,
    "intitule": "L2/L2-R/L2-INT/L2-Bdx",
    "Etudiants": [
      {
        "nom": "Naceur",
        "prenom": "Maha",
        "NumInscription": "P123456",
        "Notes": {
            "TP" : 14,
            "Projet" : 16,
            "DE" : 13
        }
      },
      {
        "nom": "Chin",
        "prenom": "Nathalie",
        "NumInscription": "P789098",
        "Notes": {
            "TP" : 17,
            "Projet" : 12,
            "DE" : 19
        }
      },
      {
        "nom": "Guillemard",
        "prenom": "Florian",
        "NumInscription": "B123456",
        "Notes": {
            "TP" : 18,
            "Projet" : 16,
            "DE" : 12
        }
      }
    ]
  }
```

Chargeons cet objet dans un fichier js dans une variable, nous pouvons accéder à ses données en utilisant « . »

L'objet JSON

- Objet global javascript
- Contient des méthodes pour :
  - interpréter du JSON
  - Convertir des valeurs en JSON

parse

- Analyser la chaîne comme JSON
- Transformer éventuellement la valeur produite et ses propriétés et renvoyer la valeur
- Syntaxe : JSON.parse(*texte*[, *revivificateur*])

stringify

- Retourne une chaîne JSON correspondant à la valeur spécifiée,
- Inclus certaines propriétés
- Remplace les valeurs des propriétés d'une manière définie par l'utilisateur.
- Syntaxe : JSON.stringify(*valeur*[, remplacement[, expace]])

Exemple

```js
var months = [ {"id":1,"name":"January"},
{"id":2,"name":"February"} ];
console.log(JSON.stringify(months));
var monthNames = ["January", "February"];
var month = {};
var monthsArray = [];
for (let i = 0; i < 2; i++) {
    month.id = (i + 1);
    month.name = monthNames[i];
    monthsArray.push({...month});
}
console.log(JSON.stringify(monthsArray));
```

```
"[{'id':1,'name':'January'},
{'id':2,'name':'February'}]"

"[{'id':1,'name':'January'},
{'id':2,'name':'February'}]"

"[{'id':1,'name':'January'},
{'id':2,'name':'February'}]"

"[{'id':1,'name':'January'},
{'id':2,'name':'February'}]"
```